

Online Examination Platform

SRS, System Design, and Delivery Plan

Version: 1.0 · **Date:** September 30, 2026 **Technology:** Node.js, Express, MongoDB/Mongoose, EJS, browser JavaScript **Audience:** Product owner, developer, QA, designer, deployment owner

This is an initial engineering baseline based on the existing code and README. Implemented features and recommendations are identified separately. This is not a production audit or legal opinion.

1. Document status and product goal

The repository contains evidence of teacher/student roles, exam creation and assignment, MCQ scoring, written-answer uploads and teacher grading, results, and AI-assisted question suggestions from PDF/image documents. This document does not assume that every behavior is production-ready.

Goal: Teachers can prepare questions and assign timed exams to selected students. Students can take exams and view permitted results and history. Teachers can review results and grade written answers.

Included in this baseline: Teacher/student experiences; session-based sign-in; MCQ and written questions; AI-assisted question drafts; exam scheduling, marks, negative marking, retakes and result policies; submissions, scoring, grading, results, analytics and export.

Separate scope: Live proctoring, AI cheating detection, payments, LMS integration, native mobile apps, high-stakes identity verification, and multi-tenant isolation are not treated as existing capabilities. They need separate discovery.

2. Users and main workflows

Role	Needs and permissions	Success signal
Student	View assigned exams, submit answers/files in time, view permitted results	Attempt is tied to the right student; timeout policy is enforced
Teacher	Create questions, assign exams, review results, grade written answers, release results	Controls only their exams; release policy is enforced
Operator (proposed)	Manage secrets, backups, monitoring and incidents	Service is recoverable and sensitive data is protected

- 1. **Prepare:** Teacher signs in → creates/imports questions → reviews and edits AI suggestions → sets exam, schedule, marks and result policy → assigns students.
- 2. **Take exam:** Student signs in → exam assignment and timing are checked → attempt begins → answers/files are saved → submit/timeout → server creates the result.

3. **Grade written work:** Teacher reviews pending answers → enters marks/feedback → score is recalculated → result is released according to policy.

Online Examination System — Engineering Blueprint v1.0 · Internal project document

4. **Review:** Teacher filters/exports results and reviews analytics; students see only their permitted results.

3. SRS — functional requirements

Priority: **M** must-have, **S** important, **C** optional/later.

ID	Requirement	Priority	Verifiable outcome
FR-01	Register/sign in; hash passwords; determine role from the server session	M	No raw password is stored; invalid login is rejected
FR-02	Enforce teacher-only actions and ownership checks on the server	M	A student cannot perform teacher actions through direct API calls
FR-03	Teachers create/edit MCQ and written questions	M	Question ownership and input validation are enforced
FR-04	Set exam schedule, marks, negative marking, retakes and result policy	M	Invalid values are rejected; total marks are consistent
FR-05	Students start only exams assigned to them and within the permitted time	M	Access using another ID or a closed window is rejected
FR-06	Save attempt answers on the server; define retry, timeout and duplicate rules	M	A retry does not create a duplicate result for the same attempt
FR-07	Calculate MCQ score, skipped answers and negative marks on the server	M	Answer key is not present in the pre-submit student response
FR-08	Accept written-answer files within MIME/size limits; allow teacher grading	S	Accepted files are linked to the attempt; marks stay within the maximum
FR-09	Enforce immediate, after-exam, or teacher-release result policies	M	Unreleased results are absent from student endpoints
FR-10	Students view their own results/history; teachers view results for their exams	M	Access using another student/teacher ID is denied
FR-11	Teachers filter results and export PDF/Excel	S	Export matches the filtered list and respects data scope
FR-12	AI proposes questions; teacher reviews before saving or publishing	S	Output is a draft; provider key remains server-side
FR-13	Enforce attempt number and concurrency according to retake policy	M	Concurrent requests preserve attempt uniqueness
FR-14	Record audit events for important publish/grade changes	S	Actor/time/entity can be identified without logging secrets

4. SRS — quality and constraint requirements

Online Examination System — Engineering Blueprint — v1.0 — Internal project document

ID	Requirement	Proposed acceptance target
NFR-01 Performance	Responsive API/dashboard under peak exam load	p95 $\leq$ 500ms under an agreed load profile (excluding AI/files); owner sets concurrency target
NFR-02 Availability	Monitoring and recovery	Approved uptime target, health check, alert and incident contact
NFR-03 Security	Authentication, authorization, validation, session and file security	Ownership checked on every protected route; secrets absent from repository/logs
NFR-04 Privacy	Data minimization, retention and deletion	Privacy notice and retention policy approved before launch
NFR-05 Integrity	Prevent duplicate or tampered attempts/results	Idempotent submission, unique indexes, atomic updates
NFR-06 Accessibility	Keyboard, labels, contrast and screen reader support	WCAG 2.2 AA target; manual review of exam workflow
NFR-07 Compatibility	Browser and mobile support	Verify current and previous major Chrome/Edge/Firefox versions
NFR-08 Maintainability	Modular code, API contract and consistent errors	Business rules in services; API schemas documented
NFR-09 Recovery	Backups and recovery exercises	RPO/RTO defined; encrypted backup restore rehearsal

Numeric targets are proposals, not production SLAs. Approve them against real traffic and business needs.

5. System architecture and data model

```

Browser (EJS + static JS/CSS)
└─ HTTPS/session → Express → auth/role middleware → routes/controllers
                                   → services (business rules)
                                   → repositories/Mongoose → MongoDB
External: Gemini API (AI question suggestions only)
  
```

The backend currently uses a modular Express/Mongoose layout. Enforce permissions, timing, scoring, result visibility and ownership on the server. Handle database startup failure explicitly.

Entity	Main data/relationships	Design notes
User (Auth)	name, normalized email, passwordHash, role, class	Verify that the service hashes passwords and prevents role escalation
Question	teacher owner, prompt, type, options/key, subject	Never return answer keys to students; check ownership for CRUD
Exam (AssignedQuestion)	teacher, students, questions, schedule, marks, policies	Decide question version/snapshot rules after publishing
Attempt/Result	exam, student, attempt no., timing, answers, score/state	Define idempotency, unique (exam, student, attempt), and source of truth

Entity	Main data/relationships	Design notes
Online Examination System · WrittenAnswer	Engineering Blueprint · v1.0 · Internal project document exam/student/question/attempt, file metadata/data	Consider size, malware scan, access, retention and object storage
AuditEvent (proposed)	actor, action, entity ID, timestamp, request ID	Never log passwords, keys, file contents or session secrets

Code review follow-up: Check that the Submission schema's student reference model and answer type align with the question flow. Choose one source of truth for legacy Submission versus Result/Attempt.

API capabilities and access policy

Capability	Access	Required rules
Authentication	Public/own session	Normalized email, rate limits, secure cookie, CSRF strategy
Question CRUD / OCR generation	Teacher + owner	Validate AI output; limit uploads and provider timeouts
Exam assign/list/start	Teacher owner / assigned student	Check schedule, duration, retake and student scope
Answer submit / file upload	Attempt owner	Server clock, idempotency, exam window, size/MIME
Results / grade / release / export	Teacher owner or permitted student	Apply visibility policy on every endpoint

Document actual paths/methods/schemas/status codes in OpenAPI after checking route and controller implementations.

6. Security, privacy, and reliability

- **Access:** Use the authenticated session ID as the actor; do not trust client-supplied IDs; authorize every object read/write.
- **Sessions/secrets:** Use a random production `SESSION_SECRET`, HTTPS, Secure/HttpOnly/SameSite cookies, logout invalidation and login rate limits. Keep `.env` out of Git and provider keys on the server.
- **Exam integrity:** Server clock is authoritative; do not expose answer keys before submission; enforce schedule/duration/retake; prevent duplicate submit atomically.
- **Files/input:** Validate schemas, cap file size, sniff MIME type, sanitize filenames, use safe download headers, and consider malware scanning or isolated storage.
- **Web:** Use CSRF defenses on session-based state-changing routes; review XSS/security headers; prevent formula injection in CSV/Excel exports.
- **Privacy:** Minimize data, audit exports, define retention/deletion, publish a privacy notice, and review consent/provider retention before sending documents to an AI provider.
- **Operations:** Use redacted structured logs, request IDs, alerts, encrypted backups, restore rehearsals and an incident runbook.

Edge cases: Define the outcome of submit versus timeout races; retries must not create duplicate results; define answer recovery after refresh/network loss; snapshot active exam questions; recover when file upload succeeds

but exam submission fails.

Online Examination System · Engineering Blueprint · v1.0 · Internal project document

7. QA and release acceptance

No tests were run while preparing this PDF. Add the following QA layers to the release pipeline:

Layer	Checks
Unit	Scoring/negative marking/skips, visibility, validation, grade boundaries, file policy
Integration	Mongo repositories, session/role/owner checks, unique indexes/idempotency
End-to-end	Teacher create→assign→student attempt→submit→grade/release→student view
Security	Unauthenticated access, IDOR, privilege escalation, CSRF/XSS, brute force, file abuse
Load/recovery	Concurrent starts/submits, DB restart, backup restore, provider failure
Accessibility	Keyboard-only exam, mobile, error messages, focus and contrast

Acceptance criteria: (1) Reproducible passing evidence for all FR-M requirements. (2) Students cannot read others' results/answers, answer keys, or unreleased results; teachers cannot change another teacher's exam. (3) Timer, score, retake and visibility are server-authoritative; no duplicate result. (4) Secure production configuration and backup/rollback are verified. (5) Known issues, browser support, retention and incident owner are documented.

8. Typical company engineering flow and delivery plan

Phase	Typical work	Deliverable/gate for this project
0 Discovery	Stakeholders, personas, scope, constraints, risks	Product owner decisions on concurrency, hosting, privacy and exam policy
1 Specification	SRS, acceptance criteria, workflows, API/data contract	Approve functional requirements and resolve ambiguous policies
2 UX/Architecture	Wireframes, threat model, data flow, ADRs	Start/submit/error/grade flows; file/retention design
3 Build	Vertical slices, code review, migrations/config	Auth → question/exam → attempt/scoring → grading/results
4 Verify	Automated QA, security/accessibility/load checks	Acceptance evidence and issue list
5 Release	Staging UAT, backups, rollout/rollback	Deployment checklist, monitoring, restore proof, named owner
6 Operate	Monitoring, incidents, feedback, patches	Latency/error/submit metrics; access and backup reviews

Suggested practice: Track user stories and acceptance criteria in issues; use small reviewed PRs; deploy dev→staging→production; add lint, automated tests and dependency/security scanning to CI; record important choices as ADRs; plan backup, rollback and migrations before release.

9. Risks and dependencies

Online Examination System — Engineering Blueprint — v1.0 — Internal project document

Risk	Impact	Mitigation
Trusting client timer/role	Time manipulation or unauthorized access	Server-side authorization/time checks; direct API security tests
Concurrent submit/retake race	Duplicate or incorrect results	Unique indexes, atomic transitions, idempotency
Answer key/student data in responses	Exam or privacy breach	Allowlist DTOs, role-aware serialization, leakage checks
Oversized/malicious files	Storage, availability and security impact	Size/MIME limits, scanning, isolated storage, retention
Incorrect AI questions	Unfair exam	Require teacher review; do not auto-publish
Unverified backup/restore	Permanent data loss	Encrypted backups and regular restore drills

Dependencies: MongoDB, hosting/TLS, Gemini API (AI import only), browser support, and domain/email if verification is added. Whether manual question creation remains available during provider outages is a product decision.

10. Decisions needed from the product owner

Decision	Why it matters	Owner
Target users and peak concurrent examinees	Hosting and load targets	Product owner
Exam window versus per-attempt timer precedence	Timeout enforcement	Product + engineering
Autosave/reconnect behavior	Fairness and data loss	Product + engineering
What students see while written grading is pending	Result/privacy policy	Academic owner
File types/size and DB versus object storage	Security, cost, backup	Engineering + operations
Retention/deletion and applicable privacy rules	Privacy obligations	Product + privacy reviewer
Who approves teacher accounts/roles	Prevent privilege escalation	Product + security

First practical steps

1. Assign owners and collect answers to the decisions above; approve SRS v1.1.
2. Compare routes/controllers with functional requirements and API contracts; create a gap list.
3. Resolve blockers around object-level authorization, score/attempt idempotency, file limits and result visibility.
4. Add automated QA, CI, backup/restore and deployment runbooks; then conduct staging UAT.

Appendix — glossary and source

Online Examination System — Engineering Blueprint — v1.0 — Internal project document

Term	Meaning
SRS	Software Requirements Specification — testable system requirements
FR / NFR	Functional / Non-functional Requirement — feature behavior / quality constraint
RBAC	Role-Based Access Control, combined with record ownership checks
Idempotency	A retry does not apply the business operation more than once
RPO / RTO	Acceptable data-loss window / recovery time target
ADR	Architecture Decision Record — reason for a design choice

Prepared from visible repository README, Express routes/controllers/services and Mongoose schemas. This is an engineering planning artifact, not legal advice, security certification, or a complete code audit.